Attenzione su grafo in un'euristica di branching appresa per SAT

GAT-Q-SAT contro Graph-Q-SAT, a parità di training set e con due controlli

Donato Meoli

Università di Pisa



L'euristica di branching di un solutore CDCL

- Un solutore CDCL decide ripetutamente il valore di una variabile, propaga, e a ogni conflitto impara una clausola e fa backjump; la scelta della decisione è lasciata a un'euristica, di solito VSIDS, che ordina le variabili con contatori di attività.
- All'inizio della ricerca i contatori dicono poco, e una decisione sbagliata costa tempo ma non correttezza: l'euristica è il posto naturale per una politica appresa.
- Graph-Q-SAT (Kurin et al., 2020) la impara con il Deep Q-Learning sul grafo bipartito della formula, e somma i messaggi di tutti i vicini di un nodo con lo stesso peso.
- Questo lavoro aggiunge l'attenzione su grafo al suo blocco centrale (GAT-Q-SAT), e si chiede dove pesare i vicini aiuti, e se il guadagno venga dall'attenzione o dai parametri in più.

Il metodo

- Il branching come processo decisionale
- Graph-Q-SAT
- GAT-Q-SAT
- I due controlli

Kurin et al., *Can Q-Learning with Graph Networks Learn a Generalizable Branching Heuristic for a SAT Solver?*
Veličković et al., *Graph Attention Networks.*

Il branching come processo decisionale

- Lo stato è la formula semplificata dall'assegnamento parziale, con le clausole imparate; un'azione è una variabile libera con un valore di verità; un passo è una decisione con la propagazione e l'analisi dei conflitti che ne seguono.
- La ricompensa è -0.1 a ogni passo e 0 alla fine, quindi massimizzare il ritorno vuol dire terminare con il minor numero di decisioni.
- DQN approssima $Q^\star(s,a) = \mathbb{E}\big[r + \gamma \max_{a'} Q^\star(s',a')\big]$ con una target network e un replay buffer uniforme, e l'euristica è $\pi(s) = \arg\max_a Q_\theta(s,a)$.
- Il numero di azioni cambia lungo un episodio e lo stato è definito a meno di permutazioni di variabili e clausole: di qui una rete su grafo, che dà un punteggio a ogni nodo variabile e si applica a formule di qualunque dimensione.

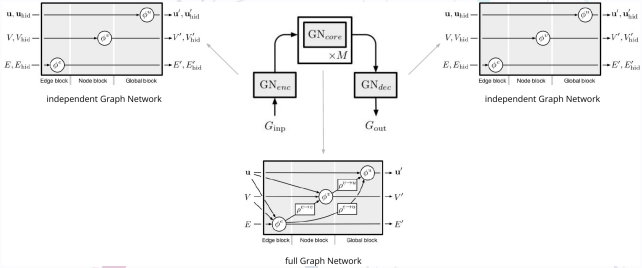
Configurazione di Kurin et al.: Adam a $2 \cdot 10^{-5}$, $\gamma = 0.99$, buffer di 20000, minibatch di 64, 50000 aggiornamenti, al più 500 decisioni del modello per episodio (il cap).

Graph-Q-SAT

Nodi: le variabili libere e le clausole non ancora soddisfatte; archi: le occorrenze, con la polarità. Un blocco centrale, applicato $M = 4$ volte con pesi condivisi, aggiorna gli archi, i nodi e l'attributo globale:

$$e'_{ij} = \phi^e(e_{ij}, v_i, v_j, u)$$
$$v'_i = \phi^v(\sum_{j \in \mathcal{N}_i} e'_{ij}, v_i, u)$$
$$u' = \phi^u(\bar{e}', \bar{v}', u)$$

La somma dà lo stesso peso a tutte le clausole di una variabile; 71144 parametri.



encoder, blocco centrale $\times M$, decoder (da Battaglia et al.)

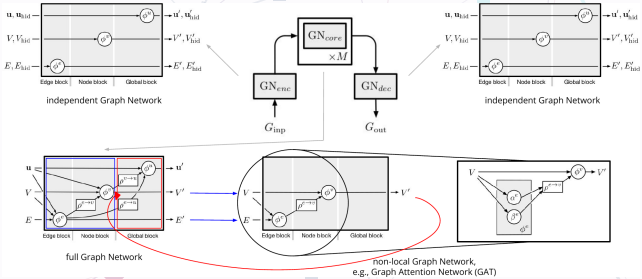
GAT-Q-SAT

Due strati di attenzione con le feature degli archi fra l'aggiornamento dei nodi e quello globale (3 teste, poi 1):

$$v_i^+ = \Big\|_k \sum_{j \in \mathcal{N}_i} \alpha_{ij}^k W_k v_j + b$$

$$\alpha_{ij}^k \propto \exp \mathrm{LeakyReLU}(\mathbf{w}_k^\top [W_k v_i \,\|\, W_k v_j \,\|\, W_k^e e'_{ji}])$$

I coefficienti dipendono dagli embedding correnti degli archi, che portano la polarità e ciò che il blocco vi ha aggiunto. 113256 parametri, circa il 60% in più.



il blocco centrale con un blocco non locale inserito

Attenzione o capacità: i due controlli

GAT-Q-SAT aggiunge l'attenzione alla somma, quindi un guadagno può venire dall'attenzione o semplicemente dalla rete più grande. Altri due modelli, con la stessa configurazione:

- Graph-Q-SAT wide: strati nascosti di 104 unità invece di 64, la capacità di GAT-Q-SAT senza attenzione;
- attention aggregation: la somma sostituita da una media pesata dall'attenzione (3 teste, mediate), alla capacità di Graph-Q-SAT.

	Graph-Q-SAT	Graph-Q-SAT wide	attention aggregation	GAT-Q-SAT
parametri	71144	113504	71627	113256
semi di training	3	2	2	3

Port e impostazione

- Il port allo stack attuale
- La misura e il solutore di riferimento

Il port, e una prova di ciò che calcola

- Il codice del 2020 è scritto per PyTorch 1.1 con estensioni compilate a versione fissa (`torch-scatter`, `torch-sparse`), che non si installano più: l'ho portato alle versioni attuali di PyTorch e PyTorch Geometric.
- Le aggregazioni vengono da `torch_geometric.utils.scatter`; l'ambiente gira con `gym` e con `gymnasium`; il wrapper di MiniSat è ricompilato su `numpy` 2; i checkpoint del 2021 si caricano con il `GATConv` attuale.
- I checkpoint del 2021, valutati sul nuovo stack, danno le stesse iterazioni del 2021 su ogni istanza in 63 valutazioni su 86, e mediane entro 0.03 nelle altre (su `flat30-60`: 1.833 e 2.000 su entrambi gli stack).
- Il training è stato verificato riaddestrando da zero i due modelli sul coloring (i semi del 2026).

La misura e il solutore di riferimento

$$\mathrm{MRIR} = \mathrm{median}\ \frac{\#\text{iterazioni di MiniSat}}{\#\text{iterazioni del modello}}$$

sulle istanze di una famiglia; un'iterazione è una decisione, del modello o di VSIDS, che subentra dopo il cap (500 se non detto altrimenti). Sopra 1 il modello prende meno decisioni di MiniSat.

- Il modello guida MiniSat senza restart; al denominatore c'è il MiniSat più forte su ogni famiglia: con i restart sul coloring (fino al 26% di iterazioni in meno), senza sul random 3-SAT uniforme (da 2.2 a 2.9 volte meno a 250 variabili).
- 100 istanze di test per famiglia; coloring da 30 a 200 nodi, addestrato su 50.

- Graph coloring, per dimensione
- Capacità o attenzione
- Un cambio della distribuzione di training
- Transfer fra domini
- Random 3-SAT uniforme
- Iterazioni e tempo

Risultati

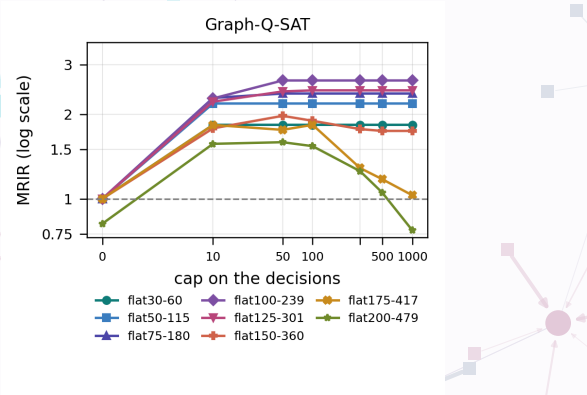
Graph coloring: il divario cresce con la dimensione

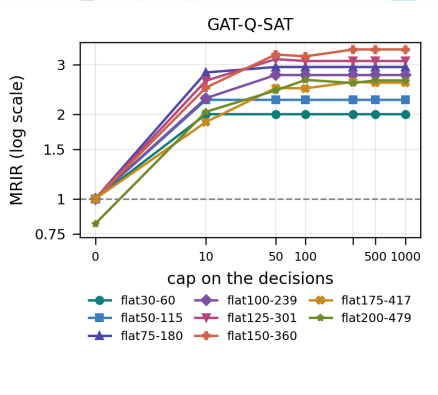
famiglia	Graph-Q-SAT	GAT-Q-SAT	Δ
flat30-60	1.83	2.00	+0.17
flat50-115	2.18	2.25	+0.07
flat75-180	2.37	2.94	+0.58
flat100-239	2.64	2.76	+0.12
flat125-301	2.43	3.09	+0.66
flat150-360	1.74	3.40	+1.66
flat175-417	1.17	2.59	+1.42
flat200-479	1.05	2.64	+1.59

MRIR al cap 500, media sui semi

- GAT-Q-SAT prende meno decisioni su tutte e otto le famiglie.
- Fino a 100 nodi i due sono vicini; da 125 nodi in su il divario è grande, soprattutto perché Graph-Q-SAT scende quasi al livello di MiniSat a 200 nodi, quattro volte la dimensione di training.

Graph coloring: l'MRIR in funzione del cap



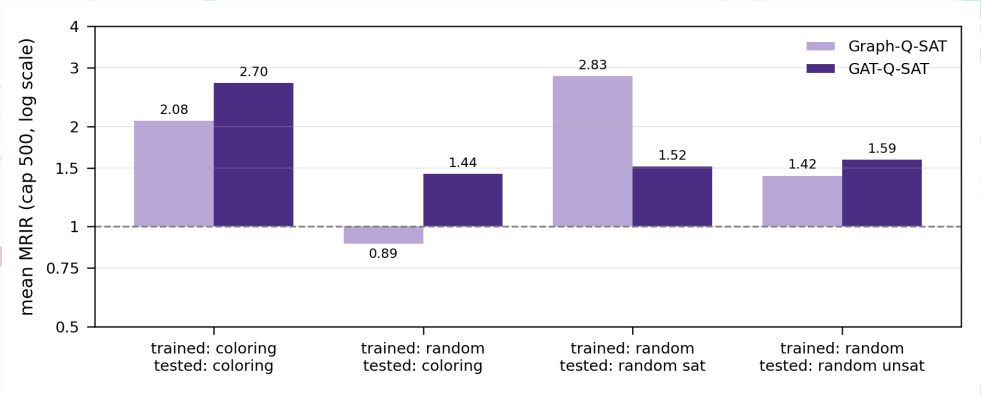


Run del 2021, una linea per famiglia, ognuna nel suo colore (lo stesso in ogni figura); il punto al cap 0 è MiniSat senza restart. Sulle famiglie grandi le decisioni di Graph-Q-SAT dopo le prime decine allungano la ricerca.

Capacità o attenzione

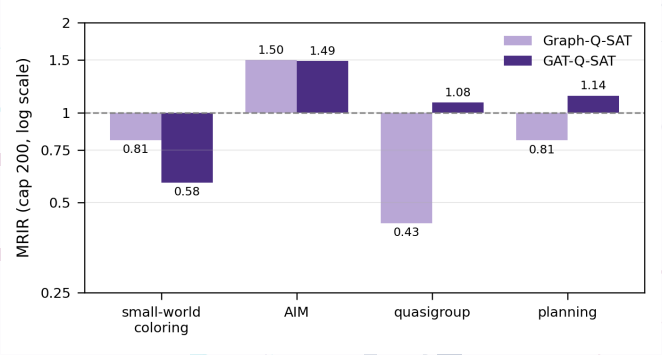
[Le quattro varianti sulle otto famiglie, dalla tabella 2 della nota: se Graph-Q-SAT wide chiude il divario, se attention aggregation arriva a GAT-Q-SAT con un sesto dei suoi parametri aggiunti, e l'ampiezza delle differenze rispetto alla variazione fra i semi.]

Addestrati su formule random, provati sul graph coloring



Lilla chiaro Graph-Q-SAT, viola scuro GAT-Q-SAT. Addestrato sul random 3-SAT uniforme, Graph-Q-SAT è sotto MiniSat su cinque delle otto famiglie di coloring, GAT-Q-SAT sopra su sette: pesare i vicini aiuta anche su una struttura non vista in training.

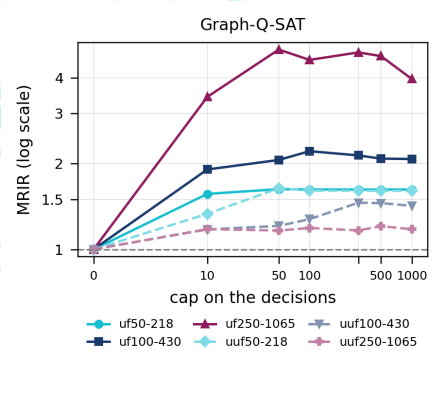
Transfer fra domini: tredici famiglie di SATLIB

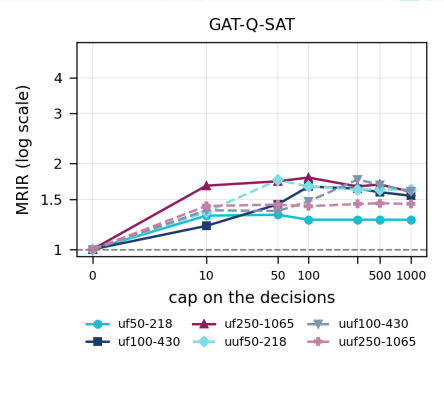


I modelli addestrati sul coloring, senza riaddestramento, su tutte le famiglie strutturate di SATLIB fino a 2000 variabili e 150000 clausole, e sul coloring small-world a nove livelli di rewiring; cap 200, MRIR contro il MiniSat più forte.

[Su quante famiglie ogni variante è sopra MiniSat e sopra le altre, e dove l'ordine si inverte.]

Random 3-SAT uniforme: l'ordine si inverte





Sulle formule soddisfacibili Graph-Q-SAT è migliore a tutte le dimensioni (4.78 contro 1.69 a 250 variabili): alla transizione di fase i vicini di un nodo sono statisticamente uguali. Sulle insoddisfacibili i due sono alla pari o GAT-Q-SAT è avanti.

Iterazioni e tempo

	cap 50		cap 500		cap 1000	
	sec	MRIR	sec	MRIR	sec	MRIR
Graph-Q-SAT	2.55	1.59	26.51	1.05	57.67	0.77
GAT-Q-SAT	3.52	2.43	12.20	2.64	12.19	2.64

`flat200-479`, secondi per istanza, mediana

- Una decisione di GAT-Q-SAT costa di più, ma ai cap più alti è il più veloce dei due, perché termina con meno decisioni.
- Entrambi impiegano secondi dove MiniSat impiega una frazione di secondo: a questa dimensione nessuno dei due è competitivo nel tempo, ed è la ragione delle euristiche ristrette (implementate, non misurate) e dell'inferenza offline.

Conclusioni e limiti

- Sul graph coloring GAT-Q-SAT ha preso meno decisioni su ogni famiglia, con un divario che cresce con la dimensione; è stato il migliore dei due anche addestrato su formule random e provato sul coloring.
- [I controlli: attenzione o capacità.]
- Sul random 3-SAT uniforme soddisfacibile Graph-Q-SAT è stato nettamente superiore, dove non c'è struttura che l'attenzione possa pesare.
- Limiti: tre semi per modello (due per controllo) non bastano per differenze di pochi decimi; nessuno dei due modelli è competitivo con MiniSat nel tempo; le istanze più grandi di SATLIB (per esempio il bounded model checking) sono oltre la dimensione che ho potuto valutare.

Codice, istanze e log: `github.com/dmeoli/GQSAT`.

Grazie per l'*attenzione*!

github.com/dmeoli/GQSAT